Energy 187 (2019) 115883

Contents lists available at ScienceDirect

Energy

journal homepage: www.elsevier.com/locate/energy

Physics-induced graph neural network: An application to wind-farm power estimation

Junyoung Park, Jinkyoo Park*

Department of Industrial and Systems Engineering, Korea Advanced Institute of Science and Technology, Daejeon, 34141, South Korea

ARTICLE INFO

Article history:
Received 30 March 2019
Received in revised form
9 July 2019
Accepted 4 August 2019
Available online 7 August 2019

Keywords:
Physics-induced graph neural network (PGNN)·Graph neural Network·Wind farm power estimation

ABSTRACT

We propose a physics-inspired data-driven model that can estimate the power outputs of all wind turbines in any layout under any wind conditions. The proposed model comprises two parts: (1) representing a wind farm configuration with the current wind conditions as a graph, and (2) processing the graph input and estimating power outputs of all the wind turbines using a physics-induced graph neural network (PGNN). By utilizing the form of an engineering wake interaction model as a basis function, PGNN effectively imposes physics-induced bias for modelling the interaction among wind turbines into the network structure. simulation study shows that the combination of a graph representation of a wind farm and PGNN produce not only accurate and generalizable estimations but also physically explainable estimations. That is, the computing and reasoning procedures of PGNN can be understood by analyzing the intermediate features of the model. We also conduct a layout optimization experiment to show the effectiveness of PGNN as a differentiable surrogate model for wind farm power estimations.

1. Introduction

Wind energy is considered one of the most effective renewable energy sources for clean energy production [1,2]. Wind farms, a collection of wind turbines on restricted spaces of land, have been utilized to convert wind energy into the electric energy. In order to increase the efficiency of electric energy generation, modelling a single turbine and applying control techniques has been employed [3,4]. However, based on the fact that the total power (energy) production of the wind farm depends on the wake interactions of the wind turbines, some of the research focuses on wake interactions among the turbines to model the power generation of wind farms [5–7]. For any given wind condition, the total power production of a wind farm is typically much less than the sum of the power-generating capacities of all turbines in the wind farm due to complex wake interactions between wind turbines. The wake generated by upstream wind turbines reduces the power productions of the downstream wind turbines by reducing wind speed and increasing the turbulence that the downstream wind turbines encounter. In order to optimally arrange turbines in a wind farm and control the wind turbines in operation, it is imperative that we understand how the wake interaction affects the power productions of all the wind turbines in a wind farm according to different wind farm layouts (i.e. the number of wind turbines and the relative locations of the turbines) and wind conditions (i.e. wind speed and direction).

To model the wake interactions between wind turbines in a wind farm and investigate their impact on power productions, researchers have employed various approaches that can be divided into three categories: (1) the application of high resolution computational fluid dynamics (CFD) to approximate the spatial and temporal evolution of turbulent flow and its impacts on the power generation of wind turbines in a wind farm; (2) the application of engineering wake models derived from first order physics, such as the law of conservation of momentum and Bernoulli's theorem, to approximate the wake propagation and its impact on wind turbines; and (3) the application of data-driven models (i.e. statistical models or machine-learning models) to approximate the direct relationships between wind flow conditions and the responses of a target wind turbine in a wind farm. Estimating of wind turbines' responses with CFD produces the most accurate results because this model tries to solve the Navier-Stokes equation governing the spatial and temporal evolution of turbulent flow. However, the computational cost of this approach is too high to be employed in a large-scale wind farm.

* Corresponding author.
E-mail addresses: junyoungpark@kaist.ac.kr (J. Park), jinkyoo.park@kaist.ac.kr (J. Park).

https://doi.org/10.1016/j.energy.2019.115883
0360-5442/

Engineering-based wake models has been proposed to approximate the spatial propagation of steady-state turbulent flow and the wake interactions among wind turbines using fundamental physics rules, such as the conservation of momentum and energy. For example, Jenson's wake model [8] describes how wind speed changes in the wake region and how the wakes generated by multiple upstream wind turbines concurrently affect the power production of downstream wind turbines. Due to the discontinuous boundary between wake and non-wake region in the model, the wind farm layout optimization approaches using the Jenson's wake model inevitably use search-based optimization algorithms, such as genetic algorithm and particle swam optimization [9–11]. The use of these algorithms makes it impossible to optimize a layout of a large-scale wind farm. Park et al. modified Jenson's wake model so that the wind speed changes continuously in the wake region [6] to realistically model the wake propagation, which in turn allows for the computation of the gradient of a target wind farm power with respect to the coordinates of the wind turbines in a wind farm. This wake model was then used to optimize the layout of a wind farm with 80 wind turbines. Although engineering-based wake models are convenient to use, the effectiveness of this approach strongly depends on the accuracy of proposed engineering models. Additionally, such models require careful calibrations with real or simulated data, which is often time-consuming.

Data-driven models have been employed to estimate the energy productions of wind turbines at a specific location in a target wind farm under certain wind conditions [12–15]. Such models aim to describe the observed responses, i.e., the energy generation or structural loads of a wind turbine, by constructing the relationships between the inputs and the outputs based on various statistical models. For example, You et al. proposed a non-linear regression model based on kernel-smoothing to reconstruct the power curve of a wind turbine in a wind farm [13]. Especially due to the recent advances in deep learning, diverse neural network architectures have been proposed to predict wind speed or wind turbine responses by considering the spatial and temporal relationships among multiple stations or wind turbines [16–21]. Such spatio-temporal models consider the spatial correlation among wind speed of different station or responses of spatially distributed wind turbines. In addition, Park et al. investigated how the characteristics of turbulent flow affect wind turbine response statistics, such as maximum load, average energy generation, and fatigue damage of various wind turbine components [22]. Although data-driven models accomplish satisfactory results, such models' applicability is highly limited because the data-driven model can be used only for the target wind turbine or wind farm which the data is actually acquired from. This in turn raises a fundamental question: if we already have response data from a target wind turbine, why do we have to estimate this target data as if we do not know it?

In order to overcome the limitations of mentioned approaches, i.e. high computational cost, calibration burden, generalization issues, we propose a physics-inspired, data-driven model that can estimate the power outputs of all wind turbines in any layout of a wind farm under any wind condition. The proposed model comprises of two parts:

- **Encoding relative positions of wind turbines and wind conditions into a graph** We designed the graph representation of wind farm to express wake-dependent distances along the wind directions as well as positions of wind turbines as edge features. By using the graph, our model can represent any wind farm with any condition in a unified way.
- **Processing the graph input and estimating power outputs of all the wind turbines** We propose a physics-induced graph neural network (PGNN) to process the wind farm graph and estimate the power outputs of all wind turbines. PGNN specifically employs a physics-induced weighting function, which follows the form of a continuous wake-deficit model [6], guiding the PGNN to capture physically plausible relations between the wind turbines under the given wind conditions. The proposed physics-induced weighting function effectively models dynamically changing wake-interacting patterns based on wind speed and direction and the turbines' relative positions.

Numerical studies using simulated wind farm data have demonstrated that the proposed approach, i.e. the combination of graph representation of a wind farm and PGNN, produce accurate estimations of all wind turbines in a wind farm of any size and layout under any wind speed and direction. The PGNN especially exhibits remarkable generalization performance on test wind farms that have more turbines than wind farms used during training, because it learns the pairwise-interactions between two turbines through the graph representation, which can be exploited to estimate teh power outcomes of larger wind farms. We also confirmed that, due to the physics-induced weight function, PGNN not only produce accurate and generalizable estimation but also physically explainable estimates. That is, one can understand computation and reasoning procedures of the data-driven model by analyzing the intermediate features of the model. Lastly, to explain how the proposed model can be utilized in practice, we demonstrate how the proposed model can be used as a surrogate model to optimize the layout of a wind farm. Since PGNN is a differentiable surrogate model, it can be used to compute the gradient of the total wind farm output with respect to location-related variables. The computed gradients can then be used to optimize the relative locations of wind turbines in a farm to maximize total wind farm power.

The contributions of this study are summarized as follows:

- We propose a graph representation of wind farm that can be effectively used to estimate the powers of wind turbines interacting in a wind farm.
- We proposed PGNN which use the graph representation of a wind farm as an input to estimate the powers of the wind turbines while capturing physically explainable interactions among turbines with a specifically designed physics-induced weight function.
- We empirically confirm that the combination of the proposed graph representation of a wind farm and PGNN produces accurate, transferable (scalable), and interpretable estimation results.
- We demonstrate that the proposed model, as a differentiable surrogate model for a simulation, can be used to optimize the layout of a wind farm of any size.

This paper is organized as follows. Chapter 2 discusses the engineering approaches used to estimate wind turbine power production in a wind farm and graph neural network (GNN) in order to explain the background of our study. Chapter 3 describes a way to represent a target wind farm as a graph input and the details of PGNN. Chapter 4 explains the training method for the proposed model, and Chapter 5 discusses the results of various estimation tasks. Chapter 6 analyzes the role of a physics-induced weight function and applications of PGNN. Chapter 7 investigates the practical applications of PGNN. Finally, Chapter 8 concludes with a summary and suggestions for future research.

2. Related Works

This section discusses two essential backgrounds: an overview

of an engineering-model-based approaches to estimate wind farm power generation and GNN, which serves as our model's computational basis.

2.1. Engineering models for estimating wind turbine power production

In a wind farm, the interaction between the wind flow and the rotating blade of an upstream wind turbine generates wake and turbulent wind flow, which adversely affect the power productions of downstream wind turbines. Due to the difficulty of modelling dynamic fluid-structure interaction, the majority of approaches employ engineering wake models that describe the steady state of wind speed under the influence of upstream wind turbines to compute the power production of a target wind turbine.

Fig. 1 illustrates a typical four-step procedure for the computation of the power production of a wind turbine:

- Constructing wake mode describing the steady-state wind speed $u(d,r)$ at the downstream wake distance d and the radial wake distance r as

$$u(d,r) = (1 - \delta u(d,r,\alpha))U \tag{1}$$

where $\delta u(d,r,\alpha)$, referred to as the wake deficit factor, quantifies the reduction of wind speed at (d,r). $\delta u(d,r,\alpha)$ given as [6]:

$$\delta u(d,r,\alpha) = 2\alpha\left(\frac{R_0}{R_0+\kappa d}\right)^2 exp\left(-\left(\frac{r}{R_0+\kappa d}\right)^2\right) \tag{2}$$

- Here, α, R_0, and κ are the induction factor, the radius of a rotor, and the surface roughness, respectively.
- Computing the averaged deficit factor $\delta\overline{u}_{ij}$ of wind turbine i due to wind turbine j as:

$$\delta\overline{u}_{ij} = \frac{1}{\pi R_i^2}\int_{q\in\mathscr{A}_i} \delta u(d_{qj}, r_{qj}, \alpha_j)\,dq \tag{3}$$

where $\mathscr{A}_i$ is the rotor sweeping area of wind turbine i, and q is a point in $\mathscr{A}_i$, R_i denotes the radius of turbine i sweeping area (the wind blade length of turbine i). We slightly overload the notation d_{qj}, r_{qj} to denote the down-stream, radial wake distances from the center of turbine j to the point q, respectively. the integration in Equation (3) is for averaging the deficit effect occurred by the turbine j over the rotor sweep area of wind turbine i.

- Computing the aggregated deficit factor $\delta\overline{u}_i$ of wind turbine i, which aggregates the influences of wakes generated by multiple upstream wind turbines in terms of the downstream wind turbine i:

$$\delta\overline{u}_i = \sqrt{\sum_{\{j|\mathbf{W}_{i,j}=1\}} (\delta\overline{u}_{ij})^2} \tag{4}$$

where $\{j|\mathbf{W}_{i,j}=1\}$ is the set of upstream wind turbines influencing the downstream wind i.

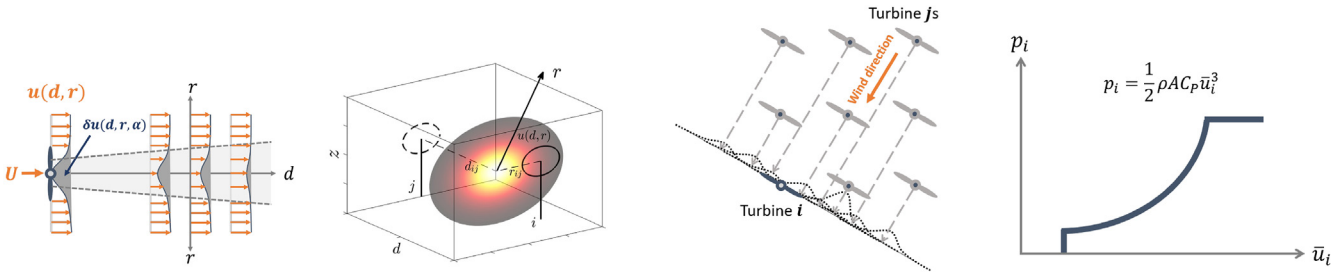
- Computing power production of wind turbine i by using the aggregated wake deficit factor $\delta\overline{u}_i$

$$p_i = \frac{1}{2}\rho A C_P \overline{u}_i^3 \text{ with } \overline{u}_i = (1 - \delta\overline{u}_i)U \tag{5}$$

2.2. Graph neural networks

GNN is a type of neural network that processes graph data. This model learns interactions among nodes and edges to extract features based on a graph's structure and the features of the nodes and edges. Among the different GNN architectures, graph convolution network (GCN) [23] (and variants) and message-passing GNN [24,25] have exhibited prominent results. GCN learns graph convolution operators that propagate node features over graph for computing updated node features. However, GCN has fundamental limitation that employing edge features in GCN computations is not straightforward while (approximately) maintaining the soundness of graph spectral theory. As a result, GCN has structural restrictions in designing graph convolution functions. On the other hand, message-passing GNN has less structural restrictions. To model the interaction among the nodes in a graph, one can freely design the routing of messages depending on the input graph structure, the functional format of message functions, and the appropriate aggregation method.

Message-passing GNN performed well in modelling and controlling physical systems. Several studies have considered the physical entity as a node of the input graph and have employed message functions to represent the interactions between nodes. The interaction network [26] infers the states of bouncing balls from raw image inputs. Modelling more complex systems such as animal and human skeleton is also possible. GNN has successfully identified different states of skeletons by considering joints as the nodes (or edge) of the graph [27,28]. By combining GNN with reinforcement learning, GNN not only performed well for robot controls, but was also transferable to novel scenarios, such as



(a) Computing the steady-state wind speed $u(d,r)$ (b) Computing the averaged deficit factor $\delta\overline{u}_{ij}$ (c) Computing the aggregated deficit factor $\delta\overline{u}_i$ (d) Computing the power production of turbine i, p_i

Fig. 1. Engineering-based wake modelling steps.

broken or additional controllable joints, without additional training steps [29].

Although GNN can consider the physics-based relationships of nodes by construction, one can impose additional regularizer on a message-passing mechanism or routing procedure of messages to learn better message function that is helpful for solving target problems. In physical system-modelling tasks, we can employ physical constraints as a regularizer. One way to impose physical constraints on a model is to augment loss function, which penalizes the network when it violates these constraints [30–32]. Seo et al. in particular employed a physics-augmented loss function to train GNN, which predicts macroscopic climate changes [31]. Even though loss function augmentation is model-agnostic, there is a clear trade-off between data-driven loss and physics-induced loss.

3. Methods

In this section, we describe the details of the proposed wind turbine power estimation model. We first discuss how to represent wind farm and wind direction information as a graph. The input graph represents the relative locations of all wind turbines in a wind farm along with wind direction and speed. The physics-induced graph neural network (PGNN) processes the input graph and estimates the power generations of all wind turbines as an output graph.

3.1. Graph representation of a wind farm

A directed graph $\mathscr{G} = (\mathscr{N}, \mathscr{E}, g)$ is a tuple that contains node features, edge features, and a global feature. Let $\mathscr{N}$, $\mathscr{E}$ be any non-ordered set of each node and edge feature where node feature $n_i \in \mathscr{N}$ is any vector that contains node-specific attributes, and edge feature $e_{ij} \in \mathscr{E}$ denotes edge attribute defined from node j to node i. We call this edge relationship *sender* j and *receiver* i [25]. $\mathscr{I}$ is an index set of all node indices. We also define index sets $\mathscr{R}_i$ and $\mathscr{S}_i$, which are the collections of all node indices *receiver*, *sender* of node i, respectively. $\mathscr{W}$ is the edge index set that contains all edge indices (i,j). Lastly, g is a vector that describe the global properties of the graph $\mathscr{G}$.

For a given wind farm layout l, the wake interaction patterns among the wind turbines change depending on wind directions. Therefore, a graph structure representing the interaction patterns among the turbines should vary according to wind directions. Fig. 2a presents an example of a constructed graph for a randomly generated wind farm with a certain wind direction. As demonstrated in Fig. 2b, the edge e_{ij} between two nodes (turbines), i and j, exists if (1) wind turbine j is the upstream turbine toward turbine i; (2) the Euclidean distance between two turbines $\delta_{ij} \leq \delta_{max}$; and (3) the contained angle $\phi_{ij} \leq \phi_{max}$. Here, δ_{max} and ϕ_{max} denote the influence cut-off distance, and angle, respectively. When turbine j satisfies three conditions with respect to turbine i, then turbine i is considered to be in the *influential region* of turbine j.

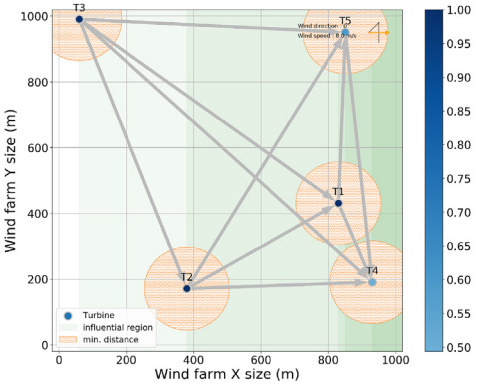
After identifying the graph structure, we can assign the node, edge, and global features for wind farm graph $\mathscr{G}$. We set n_i as global wind speed s for all turbines in the given wind farm. We also set e_{ij} as (d_{ij}, r_{ij}). Here, d_{ij} is the downstream wake-distance between two wind turbines i,j, and r_{ij} is the radial-wake distance between turbines i,j. Fig. 2b presents the graphical explanations of d_{ij}, r_{ij}. The global property g is a vector that considers graph level features. In this study, we use wind speed s as g. Empirically, it is sufficient to set n_i as wind speed s for all turbines for power estimation tasks since we assume that the turbines are homogeneous.

3.2. Physics-induced graph neural network

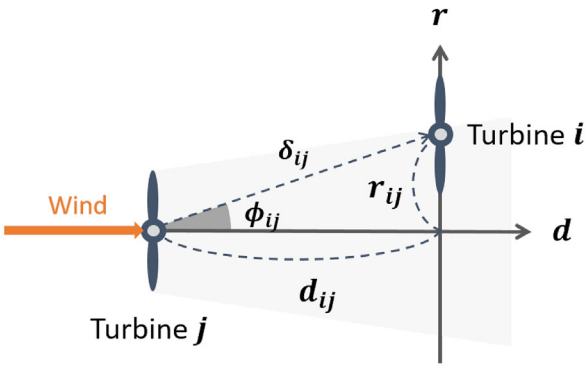
In this section, we discuss the detailed computation procedure of PGNN, the model that we propose to estimate power generations of a wind farm from the wind farm graph. PGNN is a type of message passing GNN whose message passing function and procedure are inspired by engineering wake-deficit model's functional form, and computation procedure, respectively. PGNN is composed of stacked physics-induced graph network (PGN) layers and a graph dense layer as presented in Fig. 3.

Each PGN layer update the edge, node, and graph features of the wind farm graph to be used for the estimation of the power generation of turbines by sending and updating messages among nodes of the wind farm graph. In that message-passing procedure, the PGN layer utilizes four differentiable functions: edge update function $f_e(\cdot\,;\theta_1)$, edge weight function $f_w(\cdot\,;\theta_2)$, node update function $f_n(\cdot\,;\theta_3)$, and global update function $f_g(\cdot\,;\theta_4)$. The detailed computation procedure of PGN is presented in following paragraphs.

The edge update function $f_e(\cdot\,;\theta_1)$ updates all existing edges of a graph based on Equation (6). The edge update function utilize



(a) A randomly generated feasible wind farm



(b) Graphical description of the downstream wake distances

Fig. 2. (a) Each blue dot displays randomly simulated locations of wind turbines and the corresponding powers. The orange area represents the minimum distance between turbines, and the green shadowed area represents the influential region of each turbine. The green area may vary depending on wind directions. The grey-colored arrows indicate interaction patterns between the turbines. The direction of each arrow demonstrates which turbine (potentially) affects another turbine's power generation. (b) Graphical description of the downstream wake distance d_{ij}, the radial-wake distance r_{ij}, the contained angle ϕ_{ij}, and the Euclidean distance.. (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)δ_{ij}

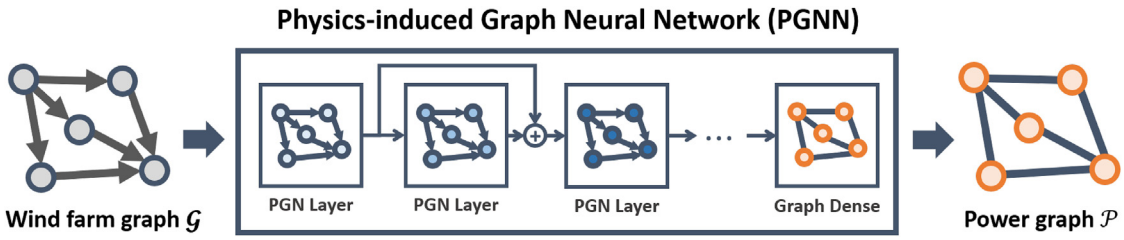


Fig. 3. An overview of a physics-induced graph neural network. PGNN comprises two distinctive building blocks: PGN layer and a graph dense layer. Roughly speaking, a series of PGN layers learn interaction patterns and features to estimate power generations. (+) denotes the residual connection. A graph dense layer works like power-conversion laws. Analogous to the traditional power-conversion laws, a graph dense layer estimates power based on the given features.

nodes feature n_j of the *sender*, node feature n_i of the *receiver*, and edge feature e_{ij} between two nodes to produce the updated edge feature e'_{ij}. The PGN layer utilizes the weight computing function $f_w(\cdot\,;\theta_2)$ to dynamically adjust weights of e'_{ij} depending on the relative position of the wind turbines and wind conditions. Here, we choose downstream wake-distance d_{ij} and radial wake distance r_{ij} as inputs of $f_w(\cdot\,;\theta_2)$ to compute weight w_{ij} that is used to adjust weight of e'_{ij}. Using the weights, PGN regulates the intensity of the influence of turbines such that the computed weight is physically make sense. The entire edge-updating procedure is presented as follows:

$$\begin{aligned} e'_{ij} &\leftarrow f_e(e_{ij}, n_j, n_i\,;\theta_1) \\ w_{ij} &\leftarrow f_w(d_{ij}, r_{ij}\,;\theta_2) \\ e'_{ij} &\leftarrow w_{ij} \times e'_{ij} \end{aligned} \tag{6}$$

The node update function $f_n(\cdot\,;\theta_3)$ updates node features using the current input node feature n_i, updated edge features e'_{ij}, and global feature g. The PGN block uses the edge-aggregating function $g_e(\cdot)$ to reduce the set of edge features into a single aggregated edge feature e'_i. The node update function $f_n(\cdot\,;\theta_3)$ then processes aggregated edge feature e'_i and the current node feature n_i to produce an updated node feature n'_i. The edge aggregation and node update routine are presented as follows:

$$\begin{aligned} e'_i &\leftarrow g_e(\mathscr{E}'_i), \text{ where } \mathscr{E}'_i = \{e'_{ji} \mid \forall j \in \mathscr{R}_i\} \\ n'_i &\leftarrow f_n(e'_i, n_i, g\,;\theta_3) \end{aligned} \tag{7}$$

The global update function $f_g(\cdot\,;\theta_4)$ updates global features. In our study, updating global property g is essential as g contains wind speed s, which is related to the total power generation of the target wind farm. PGN employs the updated edge and node features to update the global feature. To this end, PGN aggregates all edge features $\mathscr{E}'$ and all node features $\mathscr{N}'$ into two vectors $e'n'$, by utilizing $g_g(\cdot)$ and $g_n(\cdot)$. The global feature g is updated as follows:

$$\begin{aligned} e' &\leftarrow g_g(\mathscr{E}'), \text{ where } \mathscr{E}' = \{e'_{ij} \mid \forall (i,j) \in \mathscr{U}\} \\ n' &\leftarrow g_n(\mathscr{N}'), \text{ where } \mathscr{N}' = \{n'_i \mid \forall i \in \mathscr{I}\} \\ g' &\leftarrow f_g(e', n', g\,;\theta_4) \end{aligned} \tag{8}$$

In this study, $f_e(\cdot\,;\theta_1)$, $f_n(\cdot\,;\theta_3)$, and $f_g(\cdot\,;\theta_4)$ are multi-layered perceptrons (MLPs). Apart from the other encoders, we employ a physics-induced basis function to construct $f_w(\cdot\,;\theta_2)$. The edge weight function $f_w(\cdot\,;\theta_2)$ regulates the "intensity" of interactions among turbines similar to the way in which a physics-induced engineering model describes the relationships between two turbines. Here, we use the wake model proposed by Park et al. [6] as the basis function for $f_w(\cdot\,;\theta_2)$ to impose physics-induced bias on the proposed PGN:

$$f_w\left(d_{ji}, r_{ji}; \theta_2 = \left\{\alpha, R_0, \kappa\right\}\right) = \alpha\left(\frac{R_0}{R_0 + \kappa d_{ji}}\right)^2 exp\left(-\left(\frac{r_{ji}}{R_0 + \kappa d_{ji}}\right)^2\right) \tag{9}$$

By sequentially applying these updating functions on the graph input (i.e. the output graph of the previous PGN layer or wind farm graph $\mathscr{G}$), PGN produces the updated graph $\mathscr{G}' = (\mathscr{N}', \mathscr{E}', g')$. The proposed approach learns how to represent the interactions among turbines by optimizing the parameters θ_i, $i = 1,2,3,4$. Furthermore, by making all update functions to share the parameters, the proposed approach generalizes well in the estimation of power outputs for various wind farm layouts and wind conditions. The computation procedures for the PGN layer are summarized in Appendix A.

After the series of PGN layer computations, PGNN produces the updated wind farm graph $\mathscr{G}'$. The graph dense layer takes $\mathscr{G}'$ as an input and outputs power graph $\mathscr{P}$, whose node values are the estimated power outputs of the individual wind turbines in the wind farm. The graph dense layer is constructed using a single MLP, and this MLP is commonly used to estimates the power of each wind turbine in the wind farm.

We designed PGNN such that the computational procedures in PGN layers and graph dense layer mimic the computational procedures of engineering-based wake model:

- Edge weight computations in a PGN layer directly follows the computational procedure for computing the wake deficit factor $\delta u(d, r, \alpha)$ (Equation (2)).
- Edge updating steps in a PGN layer is corresponding to computing the deficit factor $\delta\overline{u}_{ij}$ between wind turbine i and j (Equation (3)).
- Edge aggregation steps in a PGN layer is corresponding to the wake deficit factor aggregation step (Equation (4)).
- Node updating steps in a PGN layer is corresponding to the computing average wind speed $\overline{u}_i$ using the aggregated deficit factor (Equation (5)).
- Estimating wind turbine power in the graph dense layer is corresponding to the procedure of the power conversion rule (Equation (5)).

4. Training procedures

4.1. Data preparation

The training data $\{\mathscr{G}_i, \mathscr{P}_i\}_{i=1,\ldots,n}$ is composed of pairs of wind farm graph $\mathscr{G}_i$ and power graph $\mathscr{P}_i$. The input graph $\mathscr{G}_i$ was constructed from a randomly generated wind farm layout and random wind conditions (wind speed, direction). The size of wind farm was chosen randomly from a set of 5, 10, 15 and 20. The layout was

randomly generated with a specified number of turbines. Wind speed s was drawn from the uniform distribution, i. e, $s \sim U(6.0\ \mathrm{m/s}, 15.0\ \mathrm{m/s})$, as well as wind direction, $\theta \sim U(0^\circ, 360^\circ)$.

For any randomly generated wind farm layout and wind condition, the power output of all the wind turbines are computed using FLORIS [33]. The computed powers are then normalized by dividing the maximum power that a turbine can produce when the turbines do not experience wake under a given wind speed. The normalized power can be perceived as an efficiency of a wind turbine under wake environment. Note that the normalized power can be easily recovered to the actual power by multiplying the actual power capacity of the wind turbine to the normalized power output. The normalized powers of all the wind turbines constitute node features of an output graph $\mathscr{P}_i$.

We prepare 900 independent training data pairs for training the proposed PGNN. For the entire training and test data, we assume that all wind turbines are homogeneous and placed on flat terrain. In addition, we further assume that each wind turbine is controlled to maximize its power output (i.e. conventional default greedy control logic).

4.2. Training physics-induced weighting function

The physics-induced weighting function $f_w(\cdot\,;\theta_2)$ takes the form of an exponential function with a set of trainable parameters, as presented in Equation (9). However, training the parameters of an exponential function is often numerically unstable because over/under-flow of computation occurs when the gradient of the proposed physics induced weighting function is computed. To alleviate the numerical instability in computing gradient, we employ a power-series approximation of exponential functions as follows:

$$f_w(d_{ji}, r_{ji}\,;\theta_2) \approx \alpha \left(\frac{R_0}{R_0 + \kappa d_{ji}}\right)^2 \sum_{k=0}^{5} \left(- \left(\frac{r_{ji}}{R_0 + \kappa d_{ji}}\right)^2 \right)^k \Big/ k! \tag{10}$$

Since the approximation error increases as the norm of input increases, we put an additional step that scales down the inputs to have smaller norm. During the training, the scaling process tracks the means and the standard deviations of downstream-wake distance d and radial-wake distance r with the exponential moving average. Then, the scale-only normalization is applied on d and r as described in Appendix B.

4.3. Hyperparameter setups

Here, we describe the hyperparameter setups for PGNN. We set the influence cut-off-angle ϕ_{max} as 90° and δ_{max} as the maximum distance between the turbines that can be placed in a target wind farm so that the proposed PGNN can learn the most general wind turbine interaction patterns. Throughout the experiments, we used three PGN layers for embedding the wind farm graph. We set every internal node, edge, and global feature as a 50-dimensional vector. All PGN layers' graph processing functions $f_e(\cdot\,;\theta_1)$, $f_n(\cdot\,;\theta_3)$, $f_g(\cdot\,;\theta_4)$ were modeled using MLPs with 256, 256, 128 dimensional hidden layers and Leaky ReLU [34] activation units.

As described above, we set $f_w(\cdot\,;\theta_2)$ to have a scale-only input normalization layer and five-degree power-series approximation for the exponential function. Residual connections exist between all PGN layers in the PGNN architecture except for the connection between the first two PGN layers. We employed one graph-dense layer to map the processed embedding vectors to power outputs of all wind turbines (i.e., power output graph). The graph dense layer has the same MLP architecture as the PGN blocks' encoding functions, and the output activation unit for the graph dense layer is ReLU. We clipped out any estimated power value larger than 1.0.

4.4. Training objective and special case treatments

We used the sum of the mean-squared-error (MSE) between the estimated power and the target power of all wind turbines in a target wind farm as a loss function for training the proposed PGNN. We employ an Adam optimizer [35] to train the proposed PGNN with learning rate $\eta = 0.001$.

In our formulation, up-stream turbines have no incoming edges. The lack of incoming edge cases make it challenging to train PGNN seamlessly. To cope with these cases, we set the incoming edge vector as $\mathbf{0}$ with the same dimension as the output of $f_e(\cdot\,;\theta_1)$. By setting virtually existing edges as $\mathbf{0}$, our PGNN handles an arbitrary number of input edges seamlessly. Additionally, we were able to interpret $w_{ij} \approx 0$ in a explainable way. Presumably, PGNN learns the same update procedure when downstream turbine i has a negligible wake-interaction with the turbine j (when $w_{ij} \approx 0$) as the upstream turbines.

5. Estimation performance analysis

The trained wind farm power estimation model is employed to estimate the powers of all wind turbines in a form of graph $\mathscr{P}$ for any number of turbines n, any layout l of a wind farm, any wind direction θ, and any wind speed s. To validate the predictive performances of the proposed model, we conducted the following experiments:

- Generalization test over wind speed and wind direction given the fixed wind farm layout
- Generalization test over wind speed and wind direction under random farm layouts

We evaluate the performances of the estimation tasks using the mean absolute percentage error, MAPE $(\mathscr{P}, \mathscr{P}')$ between the estimated (graph) power outputs $\mathscr{P}'$ and the simulated power output $\mathscr{P}$, which can be computed as follows:

$$\mathrm{MAPE}\ (\mathscr{P}, \mathscr{P}') = \frac{100\%}{n} \sum_{i=1}^{N} \left| \frac{\mathscr{P}(i) - \mathscr{P}'(i)}{\mathscr{P}(i)} \right| \tag{11}$$

where $\mathscr{P}(i)$ is the i-th node in graph $\mathscr{P}$ and N is the number of nodes (wind turbines).

For the sake of simplicity, we define the estimation function f that maps a set of input conditions (n, l, θ, s) to the power estimation output graph $\mathscr{P}$, where n is the number of wind turbines, l is the layout of n wind turbines, s is wind speed, and θ is wind direction:

$$f : \mathscr{N} \times \mathscr{L} \times \Theta \times \mathscr{S} \rightarrow \mathscr{P} \tag{12}$$

5.1. Generalization test over wind speed and wind direction given fixed wind farm layout

We first generate the random wind farm layout l_0 of five wind

turbines, as displayed Fig. 2a. We then investigate how well the trained PGNN estimates the powers of wind turbines for wind direction varying from 0° to 360° and wind speed varying from 6m /s to 15m /s. That is, we compare the following quantities to the target power graphs.

$$f(n=5, l_0, \theta, s=8\text{m/s}) \qquad \text{for } \theta = \{0^\circ, 1^\circ, \ldots, 360^\circ\}$$
$$f(n=5, l_0, \theta \in \{0^\circ, 45^\circ, 90^\circ\}, s) \qquad \text{for } s = \{6.0, 7.0, \ldots, 15.0\}\text{m/s}$$

In general, PGNN accurately estimates the power productions of wind turbines in the farm. Fig. 4 presents the estimation result that demonstrates the estimated power generations for all five wind turbines and the target power generations. The figures in the left column compare the values for various wind direction cases, while the figures on the right column compare the values for various wind speed. The average MAPE for various wind directions are 1.579%. The average MAPE for varying speed are 1.825%, 1.491%, and 2.367% when $\theta = [0^\circ, 45^\circ, 90^\circ]$, respectively.

5.2. Generalization test over wind speed and wind direction under random wind farm layout

The previous experiment considered a specific wind farm layout. Since the estimation accuracy may depend on the layout of wind turbines, we use an error measure $e(\theta, s)$, which is an average

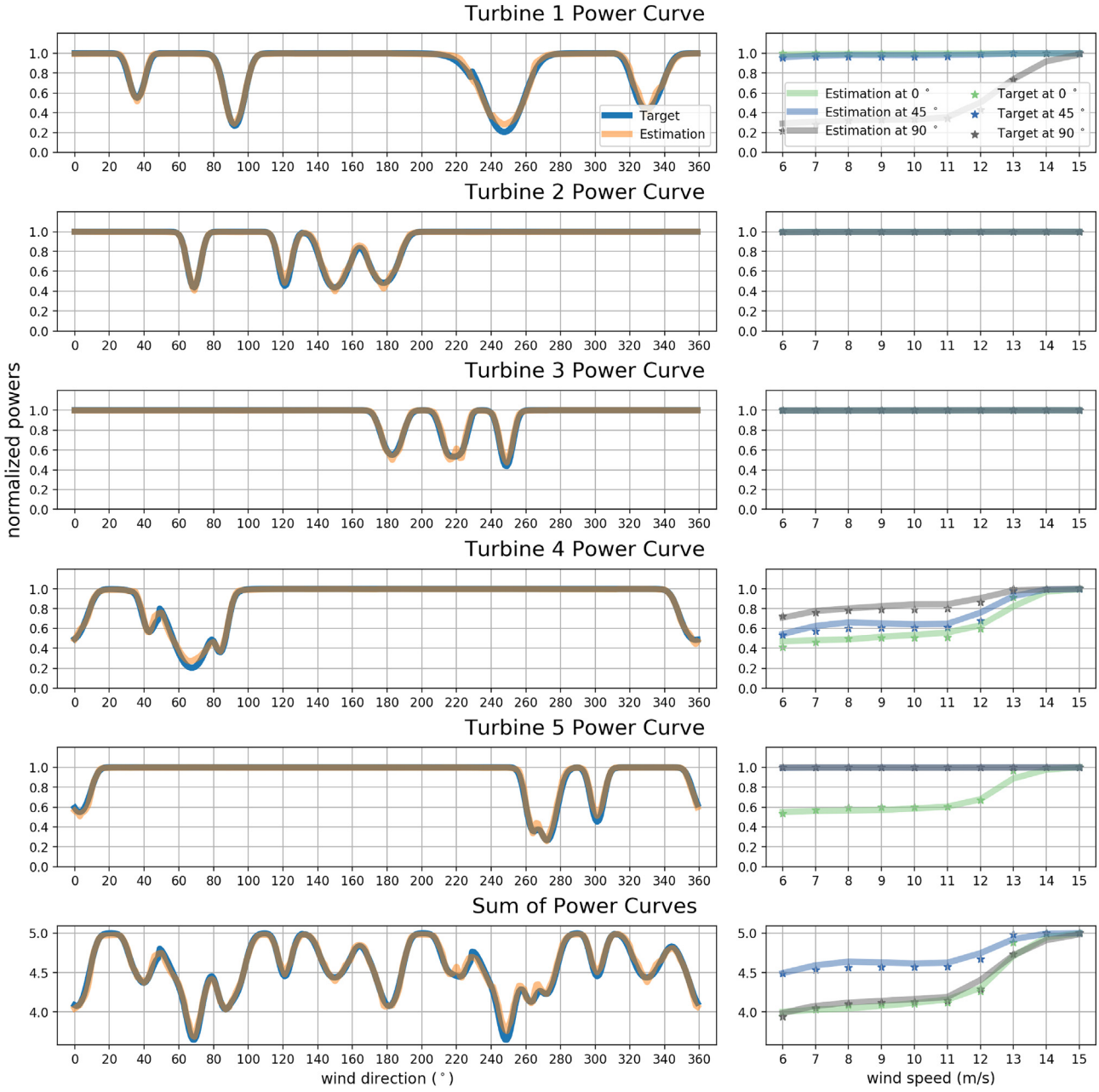


Fig. 4. Generalization performances over environmental factors: The left column displays power estimation results when wind speed s is fixed as 8.0m /s. The right column describes the power estimation result according to the wind speed s when wind direction θ is given as 0°, 45°, and 90°.

MAPE for 20 different wind-farm layouts ($N_L = 20$) of 20 wind turbines ($n = 20$) for given wind direction θ and speed s:

$$e(\theta, s) = \frac{\sum_{i=1}^{N_L} \text{MAPE}\left(\mathscr{P}_i, f(n = 20, l_i, \theta, s)\right)}{N_L} \tag{13}$$

We randomly generate wind farm layout l_i for $i = 1, 2, \ldots, 20$.

The trained PGNN estimates the powers generally well in different wind farm layouts and according to different wind speeds and directions. Fig. 5 summarizes the estimation errors for different combinations of wind direction and wind speed. Here, we set the number of turbines n as 20, which is the maximum number of wind turbines that the trained PGNN experienced during the training process. In general, it is more difficult to estimate powers as the number of wind turbines in a wind farm increase. By inspecting performance on 20-turbines farms, we can roughly evaluate the lower bound of performance within the training data distribution. The average $e(\theta, s)$ for all θ and s was 2.911%, while the maximum $e(\theta, s)$ was 6.85%. As expected, the trained PGNN reported nearly no estimation errors when wind speed was fast, since the airflow already contains enough wind energy to produce the maximal powers in turbines despite the existence of severe turbulence. That is, estimations for high-wind-speed regions is relatively simple compared to other regions.

6. Analysis of the physics-induced weighting function

The physics-induced weight function dynamically constructs a "soft" graph, assigning continuous weight values to each edge. The physics-induced weight function tends to place a larger weight on the edge between the two wind turbines that are under strong wake interaction. In this section, we analyze roles of such physics-induced weight function in achieving interpretable and scalable estimation results.

To clearly understand the roles of the physics-induced weight function, we compare its performances to another network with a data-induced wright function, called data-induced GNN (DGNN). DGNN utilizes an MLP of 256, 256, and 128 with LeakyReLU hidden activations to construct $f_w(\bullet; \theta_2)$. The output dimension of DGNN's $f_w(\bullet; \theta_2)$ is 1, similar to the PGNN for comparison. We employed ReLU for the output activation unit to produce the same output value range of PGNN's $f_w(\bullet; \theta_2)$.

6.1. Interpretability

The trained physics-induced weight function $f_w(\bullet; \theta_2)$ regulates the intensity of the edge vector (norm of the edge vector) in a physically plausible way as presented in the top row of Fig. 6. For instance, when wind direction $\theta = 0^\circ$, T2 exerts more significant impacts on T4 rather than on T3 since the radial-wake distance $r_{23} \geq r_{24}$ and $d_{23} \approx d_{24}$. When, $\theta = 90^\circ$, T1 affects T3 more than T2 does, i.e., $w_{13} \gg w_{23}$, even though $d_{13} \approx d_{23}$. More interestingly, PGNN effectively (completely) ignores the influence of wind turbines that are separated far from the target wind turbine in a radial or downstream direction, i.e., r or d is large. These cases are denoted by the grey dashed arrows. The data-induced weight function tends to assign the weights in reverse order; DGNN produces high weights when d and r are large, which does not conform the actual wake propagation.

6.2. Scalability

One desirable property of learning based models is the reliable generalization in out-of-training distributions. In order to validate the out-of-training performance of the PGNN, we compared the estimation performance of the PGNN and the DGNN for larger wind farm size (≥ 20 turbines) than either of the models saw during training. Additionally, we investigated how the number of wind turbines affects the computational time of the proposed method.

We design an error metric $e(n)$ to see how the average error varies as the size n of wind farm increases. The error metric $e(n)$ averages the MAPE of all the possible combinations of wind speed and wind direction as well as random wind farm layouts:

$$e(n) = \frac{\sum_{i=1}^{N_L} \text{MAPE}\left(\mathscr{P}_i, f(n, l_i, \theta_i, s_i)\right)}{N_L} \tag{14}$$

When n is specified, a layout l_i is randomly generated, $\theta_i \sim U(0^\circ\ 360^\circ)$ and $s_i \sim U(6.0\,\text{sm/s}, 15.0\,\text{m/s})$ for $i = 1, \ldots, N_L = 40$. For this experiment, we set the wind farm land size to be 4000 m $\times$ 4000 m, which is large enough to generate the largest layout configuration with 100 turbines.

We measured the generalization performance of the proposed PGNN and DGNN in a number of wind turbines scenarios. As displayed Fig. 7, the PGNN achieves consistently better outcomes than DGNN for all the scenarios. We considered the cases of up to 20 wind turbines as in-training cases and the region with more than 20 turbines as out-of-training cases. For in-training distributions, both PGNN and DGNN performed similarly. However, DGNN led to significantly high errors for the out-of-training cases. The average performance gap between the two models for out-of-training cases was 5.56% in $e(n)$, whereas the in-training performance gap was

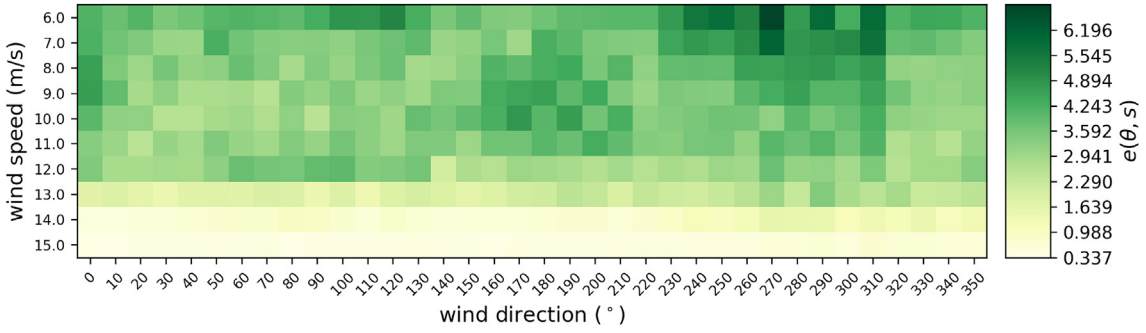


Fig. 5. Generalization over wind farm layouts: We randomly sampled 20 wind farms for each pair of wind direction and speed to measure generalization performance. We fixed the number of wind turbines n as 20. $e(\theta, s)$ are presented as a heat map.

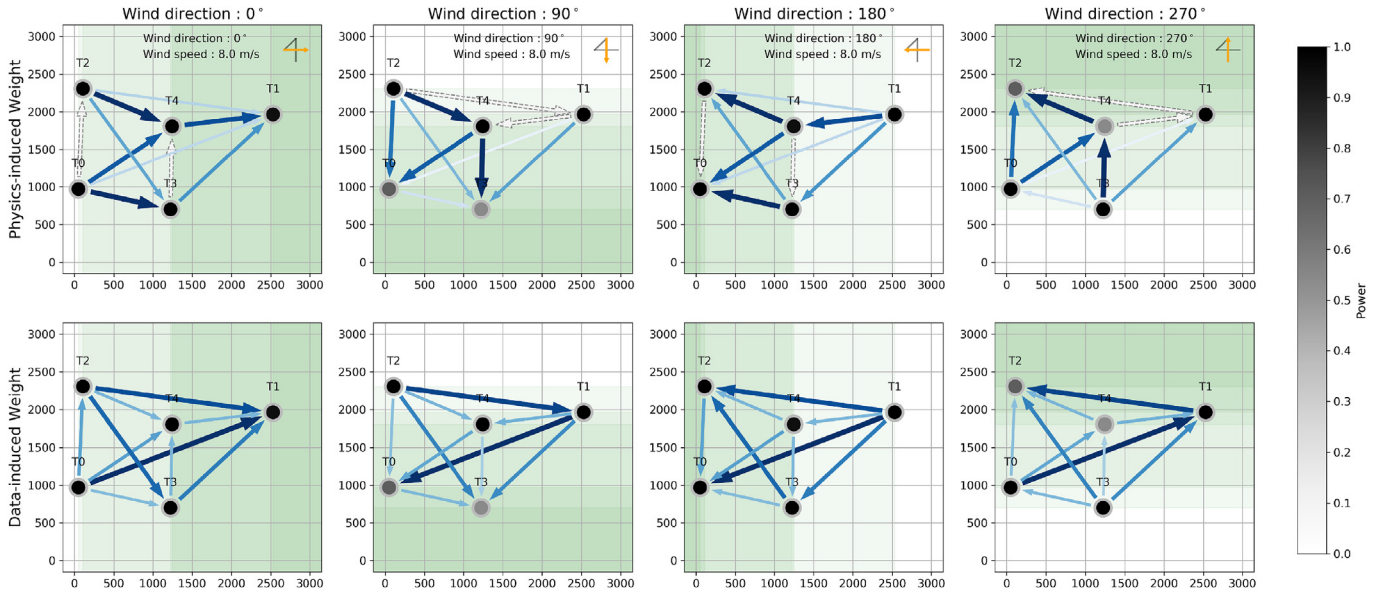


Fig. 6. : Behavior of the learned weight functions: The physics-induced weights [top row] and data-induced weights [bottom row] are visualized. Each column visualizes weights for different wind directions. The black node indicates turbine positions and corresponding powers by color gradients. The blue arrow represents the learned weight of the wind farm graphs. The direction of the arrows indicate sender-receiver relations. The grey dashed arrow is the edge with weight 0, i.e., ignored by the GNNs.

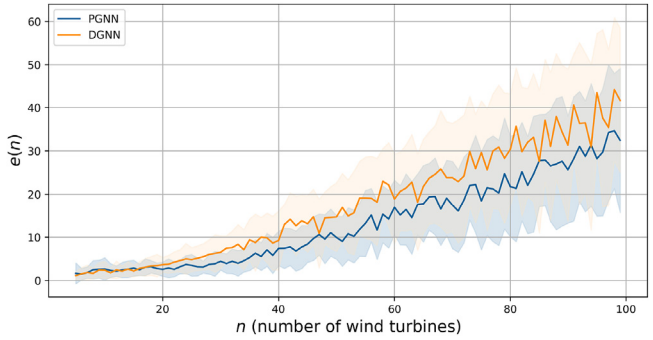


Fig. 7. Generalization test over wind farm size under random wind farm layout and wind conditions: The blue line indicates the average generalization errors of PGNN. The orange line displays that of the DGNN. Each shadow region reflects standard deviations of errors. (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

0.22%. Based on these observations, we can confirm that the physics-induced bias works as an effective regularizer in guiding PGNN to learn interactions among turbines that physically makes sense and produces accurate estimations. Thus, PGNN generalizes better for more complex scenarios.

The previous experiments demonstrate that PGNN is scalable in estimating the performance of large wind farms. Scalability in terms of computation time is also essential. Our method achieves admissible computation time on a number of wind turbines, as displayed in Fig. 8. For a generally applicable commercial wind farm size $n = 40$, it takes 0.1 s for PGNN to estimate powers of all 40 wind turbines. Even though our implementation utilized a single thread in the estimation phase, our framework is parallelizable in training or estimating the powers. With parallelization, we can reduce computational times significantly. Apart from the wall clock times, the execution time tends to increase quadratically with the number of turbines, roughly $O(n^2)$ without any parallelization. We can qualitatively analyze the results with the following facts; (1) In our graph formulation, when the number of turbines in the farm is n, our wind farm graph has n^2 edges at most; (2) Forward propagation time in the encoders are almost constant. Therefore, the PGNN has roughly $O(n^2)$ complexity with the number of wind turbines n. We can ignore the computational complexity on node updates, since nodes are linearly increasing on the number of turbines.

7. Applications

7.1. Application to a grid layout wind farm

The analyses from the previous sections demonstrate the strengths of the PGNN in arbitrary wind farm layouts and wind conditions. In order to investigate how well this model performs in a grid layout, we simulated a grid layout of 35 turbines as presented in Fig. 9. In the grid layout, the horizontal distances and vertical distances between the nodes were 5D and 7D, respectively. Here, D is the diameter of turbine rotor. On the simulated wind farm grid, we set wind speed $s = 9.0$ m/s and wind directions θ between 0° and 350° with 10° increments and then compare the estimations of PGNN and DGNN to the corresponding targets.

Fig. 9 compares the estimations of PGNN and DGNN for wind turbine 18 that experience the strong wake effect for every wind direction. Both PGNN and DGNN estimate power well when the target turbine experience simple wake interaction. i.e. when $\theta \in (0°, 90°, 180°, 270°)$. However, as the target turbine experience more complex wake interaction, DGNN tends to overestimate power generations. For instance, when $\theta \in (10° \sim 80°)$, DGNN

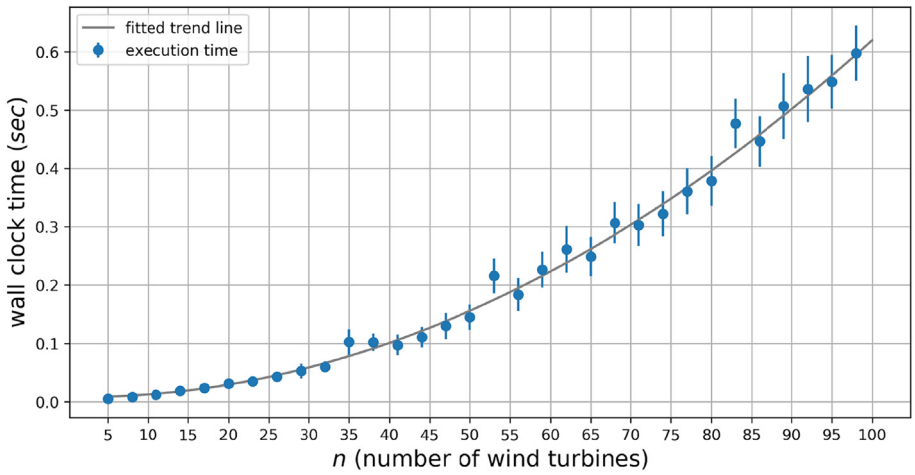


Fig. 8. Execution times of varying number of wind turbines: We generate wind farm layouts with randomly sampled wind conditions to compute execution times. For each number of wind turbines, we considered 40 realizations. The blue dot demonstrates the average execution time per one wind farm graph along the number of turbines. The blue error line indicates the standard deviations for each wind turbine cases. The light grey curve is the result of second order polynomial fitting on the execution times. We measure the computational times on a CPU machine, which is equipped with Intel(R) Core(TM) i9-7900X CPU @ 3.30 GHz. (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

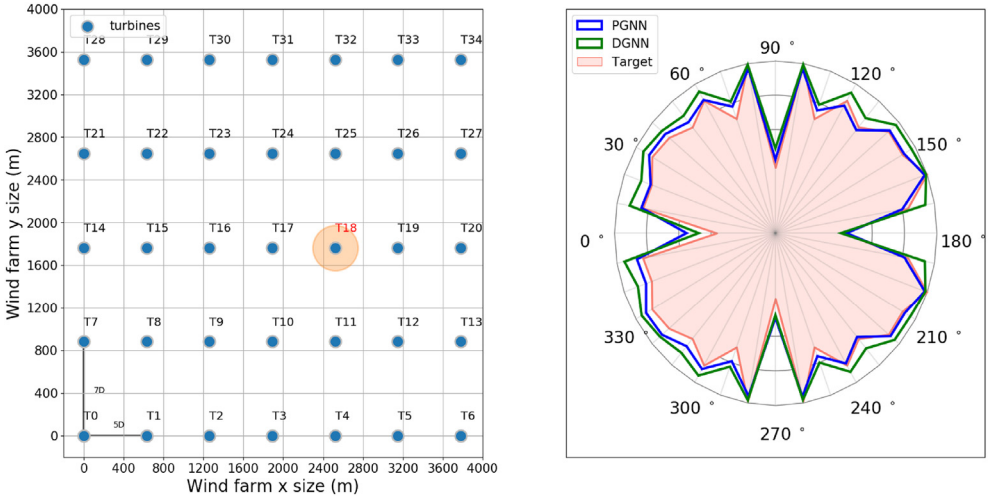


Fig. 9. Regular wind farm estimation results: The left figure presents the layout of grid wind farm. The right figure illustrates wind turbine 18 estimation results when wind speed $s = 9.0$ m/s. The blue and green line indicate estimation results of PGNN, DGNN, respectively. The red shaded area visualizes estimation target. (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

substantially overestimates the power of the target wind turbine compared to PGNN.

We also conduct similar experiments for entire wind farm. We set wind speeds as 6.0m /s, 9.0m /s, 12.0m /s. In that experiments, PGNN and DGNN achieve farm level estimation errors [8.488%, 4.287%, 5.997%] and [8.769%, 6.494%, 8.371%] for the wind speeds. We can observe the same overestimation of DGNN when target node experiences severe wake interaction. Through this experiment, we can clearly understand the effectiveness of the physics-induced weighting mechanism. The detailed results are represented in Appendix D.

7.2. Application to layout optimization

We have also demonstrated that the proposed model can be used as a surrogate model to optimize the layout of a wind farm. The objective of wind farm layout optimization is to find the relative locations of wind turbines in a wind farm in an attempt to maximize the total power generation of wind farm. In the optimization process, the trained PGNN is used to computes the partial derivative of the total power of wind farm with respect to the location-related variables, i.e., all pairs of (d_{ij}, r_{ij}) in the wind farm graph. The computed partial derivative was then used to iteratively update the layout by applying the gradient ascent method. The detailed procedure is described in appendix C.

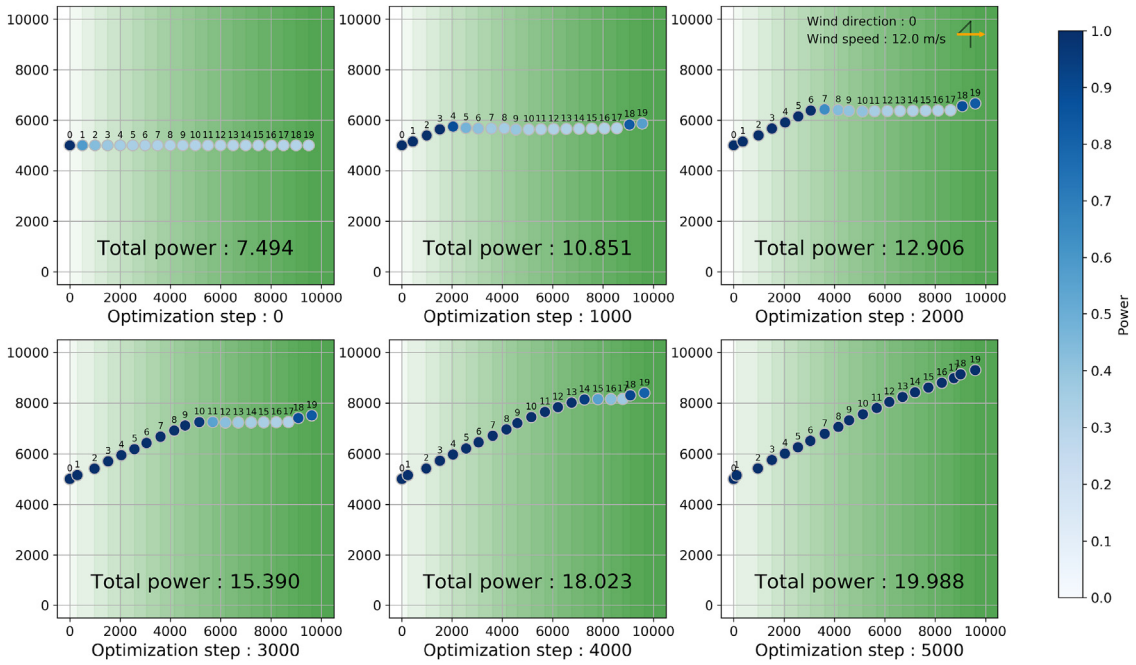


Fig. 10. Layout optimization results: The Figures demonstrate the layout evolution for different optimization steps. x, y axes indicate the grid of wind farm. The blue ball denotes the optimized turbine positions. The colors of the balls indicate power generations of turbines. (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

In this experiment, we placed 20 wind turbines in a single line to initialize the worst possible power generation scenario and set wind speed and direction as 0° and 12.0m/s, as illustrated in Fig. 10. At the initial layout, the simulated power generation is 7.494. The optimization result converges around 5000 steps with power generation of 19.988. As displayed in the figure, our optimization procedure updated the location of all turbines simultaneously, using the estimated partial derivatives from PGNN until it converged. The result implies that PGNN constructed not only the input and output relationships using data, but also embedded the correct directional information of the target wind farm power function; otherwise, the accurate derivative information would not have been acquired.

8. Conclusions

This study suggests a computational framework to estimate the power outputs of wind turbines in any form of wind farm under any wind condition. The framework is composed of two distinct parts: (1) the formation of a graph representation of the target wind farm with specific wind conditions, (2) the estimation of the power outputs of every wind turbine in the target wind farm by processing the graph representation. To process the graph represented wind farm, we propose a physics-induced graph neural network (PGNN). The proposed PGNN is composed of physic-induced graph network (PGN) layers and a graph dense layer. The PGN layer utilizes four functions (node, edge, graph feature, and physics-induced weight) to transform the input graph into the set of node embedding vectors while imposing physically plausible interaction among nodes (wind turbines). The graph dense layer then uses the node embedding vectors as inputs to estimate the power of each wind turbines.

The proposed PGNN outperforms its fully data-driven counterpart, DGNN, for all power estimation tasks. The PGNN performed significantly better when estimating out-of-training distributions compared to DGNN. Moreover, PGNN quantified the interactions among the turbines in favor of physical intuition; the DGNN, on the other hand, behaves counter intuitively. Through these observations, we can conclude that the physics-induced weight function is helpful not only for achieving better generalization but also for acquiring more interpretable results that make sense physically as well. To demonstrate more practical applications of PGNN, we have validated that PGNN estimates the powers of a large number wind turbines placed in a grid layout quite accurately. In addition, we have demonstrated that PGNN can be used as a differentiable surrogate model to optimize a layout of a large number of wind turbines.

In the future, we can consider the probabilistic version of the model for noisy data. Additionally, we can generalize graph formulation with heterogeneous turbines. Based on the expectation that the proposed framework enables us to estimate how the joint control action of wind turbines affects the outputs of the wind turbines in a wind farm, we believe that the proposed approach can be employed to control all the wind turbines in a wind farm in a cooperative way to maximize total power production.

Acknowledgement

This research was supported by Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education (2018R1D1A1B07050443).

Appendix A. Computation procedure of physics-induced graph network block

Algorithm 1
Physics-induced Graph Neural block computation.

input : set of edges features $\mathcal{E}$
set of nodes features $\mathcal{N}$
global attributes g
set of physics properties $\{d_{ij} \,|\, \forall(i,j) \in \mathcal{U}\}, \{r_{ij} \,|\, \forall(i,j) \in \mathcal{U}\}$
edge update function $f_e(\cdot\,;\theta_1)$
weight computing function $f_w(\cdot\,;\theta_2)$
node update function $f_n(\cdot\,;\theta_3)$
global attribute update function $f_g(\cdot\,;\theta_4)$
edge aggregating function $g_e(\cdot)$
global edge aggregating function $g_g(\cdot)$
global node aggregating function $g_n(\cdot)$
output : Updated graph $\mathcal{G}'$

for (i,j) ***in*** $\mathcal{U}$ **do**
  $e'_{ij} \leftarrow f_e(e_{ij}, n_j, n_i\,;\theta_1)$; // Update edge features
end
for i ***in*** $\mathcal{I}$ **do**
  for j ***in*** $\mathcal{R}_i$ **do**
    $w_{ji} \leftarrow f_w(d_{ji}, r_{ji}\,;\theta_2)$; // Compute physics-induced weight
    $e'_{ji} \leftarrow w_{ji} \times e'_{ji}$; // Regulate edge features
  end
  $\mathcal{E}'_i \leftarrow \{e'_{ji} \,|\, \forall j \in \mathcal{R}_i\}$ $e'_i \leftarrow g_e(\mathcal{E}'_i)$; // Aggregate edge features
  $n'_i \leftarrow f_n(e'_i, n_i, g\ ;\theta_3)$; // Update node features
end
$\mathcal{E}' \leftarrow \{e'_{ij} \,|\, \forall(i,j) \in \mathcal{U}\}$, $\mathcal{N}' \leftarrow \{n'_i \,|\, \forall i \in \mathcal{I}\}$;
$e' \leftarrow g_g(\mathcal{E}')$, $n' \leftarrow g_n(\mathcal{N}')$; // Aggregate updated edge, node features
$g' \leftarrow f_g(e', n', g\,;\theta_4)$; // Update global features
$\mathcal{G}' \leftarrow (\mathcal{N}', \mathcal{E}', g')$; // Assign updated graph

Appendix B. Input normalization for stable training

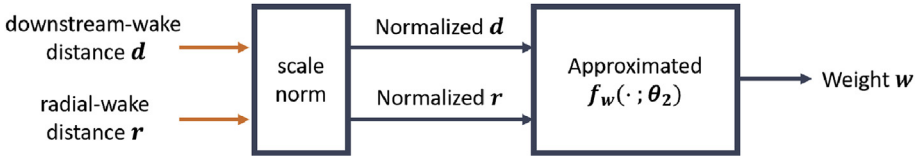


Fig. 11. Computational steps of physics induced weight function: Downstream-wake distance *d* and radial-wake distance *r* go through the "scale norm" layer. The normalized distances are fed into the physics induced weighting function to compute final weight value *w*.

Algorithm 2
Scale-only Input Normalization. 3

input : value to be normalized x
weight decreasing factor α
mean estimates $\hat{\mu}(x)$
standard deviation estimates $\hat{\sigma}(x)$
inverse scaling parameter s (learnable)
output : scale-only normalized value x'
updated mean estimates $\hat{\mu}(x)$
updated standard deviation estimates $\hat{\mu}(x)$

$\hat{\mu}(x) \leftarrow \alpha \times \hat{\mu}(x) + (1-\alpha) \times x$; // Update mean with exponential moving average
$\hat{\sigma}^2(x) \leftarrow \alpha \times \hat{\sigma}^2(x) + (1-\alpha) \times (x - \hat{\mu}(x))^2$; // Update variance
$x' \leftarrow \frac{x}{\hat{\sigma}(x)} \times \max(0, s)$; // Scaling-only normalization

Appendix C. Details on wind farm layout optimization

Algorithm 3
Layout optimization. 4

input : initial turbine coordinate $(x_i, y_i)\ \forall i \in \mathcal{I}$,
initial wind farm graph $\mathcal{G}$,
wind direction θ,
number of updates T,
step size η
output : updated turbine coordinates $(x_i, y_i)\ \forall i \in \mathcal{I}$

for $t \leftarrow 1$ ***to*** T **do**
  $\mathcal{P} \leftarrow \mathrm{PGNN}(\mathcal{G})$; // Estimate power generations
  $p = \sum_{i=1}^{\mathcal{I}} p_i$; // Compute total power
  $\frac{\partial p}{\partial d_i} = 0\ \frac{\partial p}{\partial r_i} = 0\ \forall i \in \mathcal{I}$; // Prepare 0 vectors for updating relative positions
  for (i, j) ***in*** $\mathcal{E}$ **do**
    $\frac{\partial p}{\partial d_i} \leftarrow \frac{\partial p}{\partial d_i} + \frac{\partial p}{\partial d_{ij}}\ \ \frac{\partial p}{\partial r_i} \leftarrow \frac{\partial p}{\partial r_i} + \frac{\partial p}{\partial r_{ij}}$; // Aggregate partial derivatives along the incoming edges
  end
  for i ***in*** $\mathcal{I}$ **do**
    $\begin{bmatrix}\Delta x_i \\ \Delta y_i\end{bmatrix} \leftarrow \begin{bmatrix}\cos(-\theta) & -\sin(-\theta) \\ \sin(-\theta) & \cos(-\theta)\end{bmatrix} \begin{bmatrix}\frac{\partial p}{\partial d_i} \\ \frac{\partial p}{\partial r_i}\end{bmatrix}$;
    $\Delta x_i \leftarrow \frac{\Delta x_i}{\sqrt{\Delta x_i^2 + \Delta y_i^2}}\ \ \Delta y_i \leftarrow \frac{\Delta y_i}{\sqrt{\Delta x_i^2 + \Delta y_i^2}}$; // Normalize the norm of $\Delta(x, y)$
    $x_i \leftarrow x_i + \eta \Delta x_i\ \ y_i \leftarrow y_i + \eta \Delta y_i$;
  end
  $\mathcal{G} \leftarrow$ Assign the updated coordinates $(x_i, y_i)\ \forall i \in \mathcal{I}$
end

Appendix D. Experiment result of grid wind farm estimation

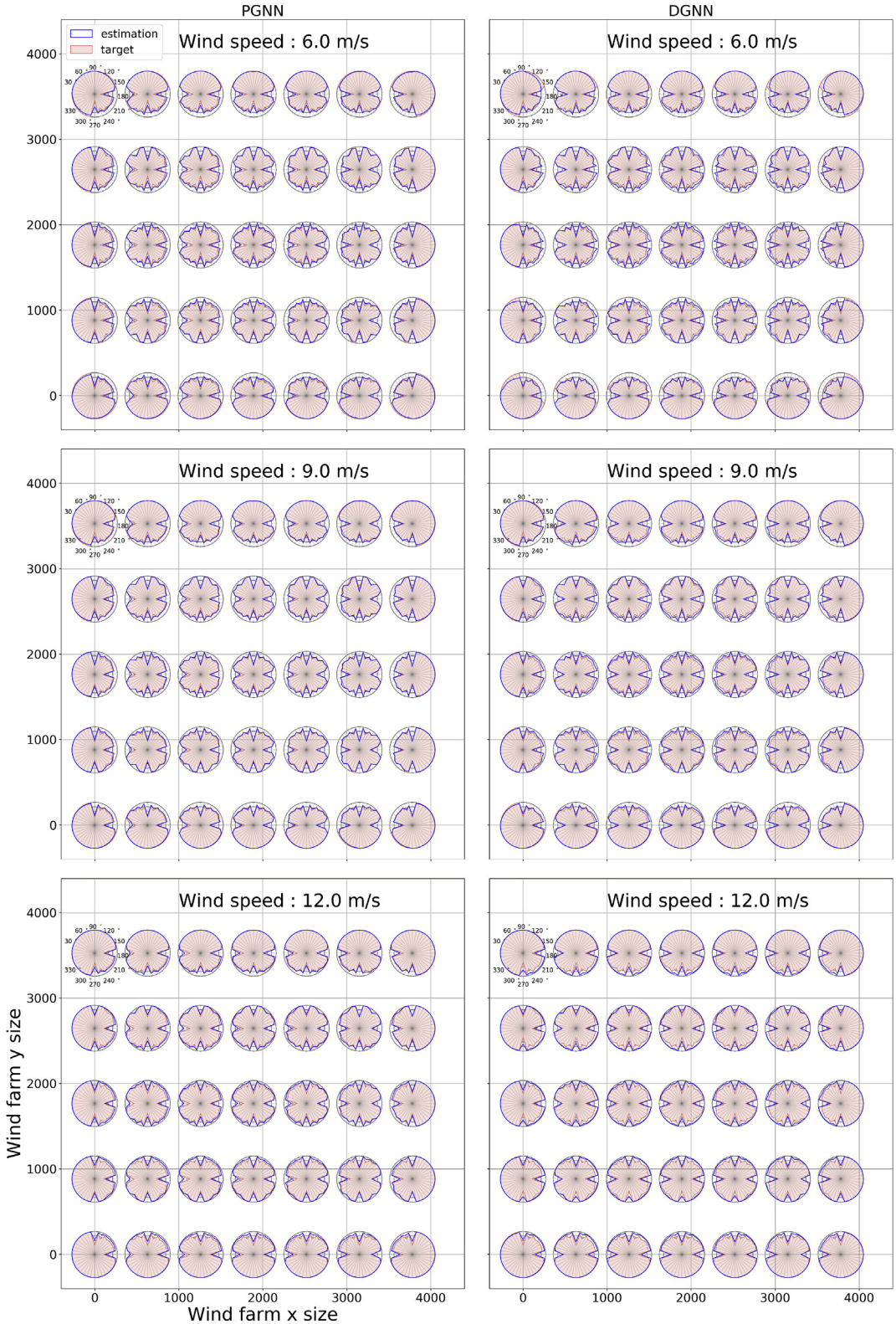


Fig. 12. Regular wind farm estimation results: the left and right columns of the figure displays the estimation results of PGNN and DGNN, respectively. We set wind speeds as 6.0m /s, 9.0m /s, 12.0m /s and wind directions for each subplot between 0° and 350° with 10° increments. The center of each polygon plot indicates the turbine position. The blue line and red shaded areas represent estimations and the corresponding targets.5

Appendix E. Supplementary data

Supplementary data to this article can be found online at https://doi.org/10.1016/j.energy.2019.115883.

References

[1] Wiser R, Lantz E, Mai T, Zayas J, DeMeo E, Eugeni E, Lin-Powers J, Tusing R. Wind vision: a new era for wind power in the United States. Electr J 2015;28(9):120–32.
[2] Abolhosseini S, Heshmati A, Altmann J. A review of renewable energy supply and energy efficiency technologies. 2014.
[3] Boukettaya G, Naifar O. Improving grid connected hybrid generation system supervision with sensorless control. J Renew Sustain Energy 2015;7(4):043115.
[4] Naifar O, Boukettaya G, Ouali A. Robust software sensor with online estimation of stator resistance applied to wecs using im. Int J Adv Manuf Technol 2016;84(5–8):885–94.
[5] Park J, Kwon S, Law KH. Wind farm power maximization based on a cooperative static game approach. In: Active and passive smart structures and integrated systems. vol. 8688. International Society for Optics and Photonics; 2013. 86880R. 2013.
[6] Park J, Law KH. Layout optimization for maximizing wind farm power production using sequential convex programming. Appl Energy 2015;151:320–34.
[7] Kusiak A, Song Z. Design of wind farm layout for maximum wind energy capture. Renew Energy 2010;35(3):685–94.
[8] Jensen NO. A note on wind generator interaction. 1983.
[9] Kusiak A, Song Z. Design of wind farm layout for maximum wind energy capture. 2010.
[10] González JS, Rodriguez AGG, Mora JC, Santos JR, Payan MB. Optimization of wind farm turbines layout using an evolutive algorithm. Renew Energy 2010;35(8):1671–81.
[11] Pookpunt S, Ongsakul W. Optimal placement of wind turbines within wind farm using binary particle swarm optimization with time-varying acceleration coefficients. Renew Energy 2013;55:266–76.
[12] Woo S, Park J, Park J. Predicting wind turbine power and load outputs by multi-task convolutional lstm model,. In: 2018 IEEE power & energy society general meeting (PESGM). IEEE; 2018. p. 1–5.
[13] You M, Liu B, Byon E, Huang S, Jin JJ. Direction-dependent power curve modeling for multiple interacting wind turbines. IEEE Trans Power Syst 2018;33(2):1725–33.
[14] Liu Z, Gao W, Wan Y-H, Muljadi E. Wind power plant prediction by using neural networks,. In: 2012 IEEE energy conversion congress and exposition (ECCE). IEEE; 2012. p. 3154–60.
[15] Shokrzadeh S, Jozani MJ, Bibeau E. Wind turbine power curve modeling using advanced parametric and nonparametric methods. IEEE Trans Sustain Energy 2014;5(4):1262–9.
[16] Noorollahi Y, Jokar MA, Kalhor A. Using artificial neural networks for temporal and spatial wind speed forecasting in Iran. Energy Convers Manag 2016;115:17–25.
[17] Ghaderi A, Sanandaji BM, Ghaderi F. Deep forecast: deep learning-based spatio-temporal forecasting. 2017. arXiv preprint arXiv:1707.08110.
[18] Cao J, Farnham DJ, Lall U. Spatial-temporal wind field prediction by artificial neural networks. 2017. arXiv preprint arXiv:1712.05293.
[19] Zhu Q, Chen J, Shi D, Zhu L, Bai X, Duan X, Liu Y. Learning temporal and spatial correlations jointly: a unified framework for wind speed prediction. *IEEE Transactions on Sustainable Energy*; 2019.
[20] Dowell J, Weiss S, Infield D. Spatio-temporal prediction of wind speed and direction by continuous directional regime. In: *Probabilistic methods Applied to power systems (PMAPS)*, 2014 international conference on. IEEE; 2014. p. 1–5.
[21] Ezzat AA, Jun M, Ding Y. Spatio-temporal asymmetry of local wind fields and its impact on short-term wind forecasting. IEEE Trans Sustain Energy 2018;9(3):1437–47.
[22] Park J, Manuel L, Basu S. Toward isolation of salient features in stable boundary layer wind fields that influence loads on wind turbines. Energies 2015;8(4):2977–3012.
[23] Kipf TN, Welling M. Semi-supervised classification with graph convolutional networks. 2016. arXiv preprint arXiv:1609.02907.
[24] Hamilton W, Ying Z, Leskovec J. Inductive representation learning on large graphs. In: Advances in neural information processing systems; 2017. p. 1024–34.
[25] Battaglia PW, Hamrick JB, Bapst V, Sanchez-Gonzalez A, Zambaldi V, Malinowski M, Tacchetti A, Raposo D, Santoro A, Faulkner R, et al. Relational inductive biases, deep learning, and graph networks. 2018. arXiv preprint arXiv:1806.01261.
[26] Battaglia P, Pascanu R, Lai M, Rezende DJ, et al. Interaction networks for learning about objects, relations and physics. In: Advances in neural information processing systems; 2016. p. 4502–10.
[27] Sanchez-Gonzalez A, Heess N, Springenberg JT, Merel J, Riedmiller M, Hadsell R, Battaglia P. Graph networks as learnable physics engines for inference and control. 2018. arXiv preprint arXiv:1806.01242.
[28] Zhang H, Ouyang H, Liu S, Qi X, Shen X, Yang R, Jia J. Human pose estimation with spatial contextual information. 2019. arXiv preprint arXiv:1901.01760.
[29] Wang T, Liao R, Ba J, Fidler S. Nervenet: learning structured policy with graph neural networks. 2018.
[30] Raissi M, Perdikaris P, Karniadakis GE. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. J Comput Phys 2019;378:686–707.
[31] Seo S, Liu Y. Differentiable physics-informed graph networks. 2019. arXiv preprint arXiv:1902.02950.
[32] Wu J-L, Kashinath K, Albert A, Chirila D, Xiao H, et al. Enforcing statistical constraints in generative adversarial networks for modeling chaotic dynamical systems. 2019. arXiv preprint arXiv:1905.06841.
[33] Floris. version 0.4.0. https://github.com/WISDEM/FLORIS/; 2018.
[34] Maas AL, Hannun AY, Ng AY. Rectifier nonlinearities improve neural network acoustic models," in Proc. icml, vol. 30; 2013. p. 3.
[35] Kingma DP, Ba J, Adam. A method for stochastic optimization. 2014. arXiv preprint arXiv:1412.6980.